Source: docs\CATEGORIES.md | Export: 11 November 2026

# Approved Stitch Categories interface - 24 October 2026

The current Categories screen uses the owner-approved desktop table, mobile cards, right-hand editor drawer and mobile bottom sheet. This section supersedes the older list/dialog layout notes below; the existing repository, schema, rules and slug reservations are unchanged.

## Current files and behavior

- `src/admin/AdminCategories.tsx`: real category list, full-list name search, enabled/disabled filters, accurate totals, existing display-order sorting, public links, edit and enable/disable actions, loading/error/empty/no-results states. It re-exports the shared editor so Products imports stay compatible.
- `src/admin/categories/CategoryEditor.tsx`: shared add/edit drawer/sheet, validated name/description/order, order stepper, staged visibility, generated/permanent link, Copy Link, failed-save retention and duplicate-submit protection. The allocated create ID remains fixed for safe retries. Renaming keeps the original slug and reservation. Existing timestamp conflict checks reject stale edits without replacing the form.
- `src/admin/categories/VisibilityConfirm.tsx`: reusable confirmation before a live category is disabled, from either the list or editor Save. Cancel leaves live visibility untouched. Enabling never publishes drafts.
- `src/admin/categories/categories.css`: category-only styling using the existing locally hosted Lora font, maroon/ivory/gold theme and compact baseline. The shared shell, 192px sidebar and existing header behavior are unchanged; no second density reduction, CDN, simulated frame or scaling is added.
- `scripts/category-browser.test.py`, `categories-fixture-control.ts`: guarded loopback-only functional/visual checks and synthetic linked products/concurrent-edit fixtures. Existing Products quick-create assertions use the shared editor's Create Category action; the recovery browser check expects the new Categories heading.

Successful writes use the same `createCategory` / `editCategory` operations and live category subscriptions. They also emit the existing administrator change signal for affected views. Category saves write no products, stock, assignments, orders or customer records. The editor captures its initial snapshot once, so background category/auth refreshes cannot overwrite dirty fields. Discard prompts occur on actual Cancel/close/navigation, not focus or visibility changes. Native dialogs include focus containment/restoration, Escape handling, scroll locking, independently scrollable content and a persistent footer on small/short viewports.

Both administrator requirements remain enforced by the unchanged Firestore rules: `admin: true` and an enabled server-managed access record. The UI offers no Delete control; deletion remains denied by the existing rules.

## Necessary reference adjustments

- Keep the approved compact shared shell and Lora rather than the prototype's alternate shell/fonts.
- Omit prototype simulators, unsupported emblem uploads, fictional per-category product counts, certification/heritage labels and unrelated bottom navigation.
- Preserve existing 0-9999 display-order validation and name/ID tie-breaking, rather than implying that a number grants a special Primary Feature placement.
- Description remains optional, at most 240 characters on one line; the textarea provides visual wrapping but does not change the schema.
- Generate links from the actual current site origin and `/collections/{slug}`, not an assumed custom domain. Creation includes an optional Customize generated link control for duplicate or non-Latin-name recovery. Editing exposes only the read-only permanent link.
- Visibility choices in the editor are staged until Save; an existing enabled category requires confirmation before a disabling write. No product is silently published/unpublished.

## Validation and local acceptance

Completed locally: typecheck, all 96 existing unit tests and production build passed. Two focused emulator suites passed: category persistence/authorization/unique reservations/deletion protection and product CRUD/category visibility/security. The Categories browser suite and full Products browser suite passed together, including Products -> Create Category -> refresh/select without lost inputs/uploads.

Category browser checks cover creation and duplicate submissions, duplicate-link recovery, copy/permanent URLs, renaming, ordering, search/status/clear/no-results, empty state, failed-commit retry, stale concurrent edits, both disable entry points and cancellation, re-enabling without publishing drafts, and unchanged product-record fingerprints. Focus/Escape, actual-discard prompts, background focus/visibility preservation, body scroll-lock cleanup and a 390x480 short viewport passed. Screenshots/overflow checks cover 320, 390, 768, 1024, 1440 and 1920px at normal zoom; desktop/mobile list, drawer/sheet and confirmation were visually compared with the references. Screenshots use only synthetic emulator data under ignored `.cache/categories-stitch/`.

Local acceptance: use the existing emulator harness and open `/admin/categories`. Search/filter/clear, add a category, edit its name/order and copy the unchanged link. Cancel a disable prompt, then confirm it and re-enable; only visibility should change. From a product editor, Create Category, save/assign it and confirm the entered product remains intact. Close a dirty sheet once with discard refused, then accept discard; switching tabs must preserve the form.

Release requires only the normal owner-controlled application release. No new rules/indexes, production preparation or dependencies are required. Hosted acceptance remains separate from local fixtures. Nothing was committed, pushed, deployed, published or changed in production during this task.

---

# Firestore categories - 17 October 2026

Subsequent product milestone: PRODUCTS.md connects real product CRUD, canonical `categoryId`, inventory and verified Cloudinary media. It supersedes this document's statements that products are unconnected. Category management and deletion denial remain intact; disabling a category hides its published products too.

Implemented locally for `parinaya-silks`, Firestore `(default)`. No live records, rules, indexes or Vercel deployment were changed. Products and other business modules remain unconnected. This guide supersedes older statements that every business module is unavailable or every browser write is denied.

## Files

| Files | Responsibility |
|---|---|
| `src/categories/model.ts`, `repository.ts`, `useCategories.ts` | Typed model, validation, server-backed subscriptions, transactional writes, slug reservation and retry. |
| `src/admin/AdminCategories.tsx` | Real list and mobile dialog; create/edit, status, ordering and confirmed-save feedback. |
| `src/admin/AdminPanel.tsx`, `AdminLayout.tsx`, `accessPolicy.ts` | Protected category route, sidebar and Products & Inventory → Manage Categories. |
| `src/services/firebase/database.ts`, `adminAuth.ts` | Shared full Firestore client; existing Auth/permission checks retained. |
| `src/App.tsx`, `src/views/ShopView.tsx`, `src/types/index.ts` | Enabled-only category navigation, stable collection URLs and future canonical product reference. |
| `firestore.rules` | Trusted validation, paired creation, uniqueness, visibility and deletion denial. |
| `scripts/category-rules.test.ts`, `category-browser.test.py`, `admin-emulator-suite.ts`, `src/categories/model.test.ts` | Emulator persistence/security, rendered browser and model coverage. |
| `scripts/storefront-browser.test.py`, `browser-audit.py`, `src/admin/accessPolicy.test.ts`, `package.json` | Regression coverage and test wiring. |

## Model

`categories/{id}` uses a stable 20-character Firestore auto ID, allocated once per create dialog. The ID is not redundantly stored as a field.

| Field | Firestore value / constraint |
|---|---|
| `name` | Trimmed string, 2-80 characters, no ASCII control characters. |
| `slug` | 1-80 lowercase ASCII letters/digits, separated by single hyphens. Immutable. |
| `description` | Optional single-line string, at most 240 characters; empty string accepted. Public when enabled. |
| `enabled` | Boolean; new forms default to disabled. |
| `displayOrder` | Integer 0-9999, ascending; ties sort by name then stable ID in the client. |
| `createdAt`, `updatedAt` | Server timestamps; creation time immutable; updates require a fresh server timestamp. |

Unknown fields are rejected on writes. Client timestamps convert to milliseconds for version comparison. Names generate a Latin slug; review it before saving. Names without ASCII letters/digits require a manually entered valid slug. Renaming preserves the ID and `/collections/{slug}`. Slug changes/reuse are unavailable.

`categorySlugs/{slug}` contains exactly `categoryId` and server `createdAt`. Creation writes the category and reservation in one transaction. Rules require both new documents to point to each other using `getAfter`; duplicate, orphan or repointed reservations are rejected independently of UI validation. Reservations cannot be updated/deleted. Retrying the same ID with an identical payload returns the persisted category without creating another record. Edits compare the loaded update timestamp to detect concurrent edits.

## Access and storefront

Category writes require BOTH the literal `admin: true` claim and `adminAccess/{uid}.enabled == true`. The shared browser SDK and deployed rules enforce this policy. No UID exception, browser-storage role, new credential or new environment variable is required.

Enabled administrators may list every category and read reservations. Signed-out users and ordinary customers may read only categories whose `enabled` is true, including direct gets. Public queries explicitly filter `where('enabled', '==', true)`; rules do not filter unrestricted queries. Reservations remain administrator-only. Existing access-record and other private collection protections remain intact.

`/shop` and `/collections` use enabled categories in the existing controls. `/collections/{slug}` resolves against the enabled-only list. Disabled/unknown URLs show Category not found. A real enabled category without products shows No products available yet. Loading, error/retry and empty states are separate; cached/pending snapshots are not shown as confirmed server results. No sample fallback or seed exists.

No composite index is needed: a single enabled equality filter uses the normal automatic index, with client sorting. If the live database manually exempts that field from indexing, restore its automatic index before acceptance.

References: Firebase atomic-write validation, transactions and query rules.

## Deletion dependency

Deletion is unavailable for ALL categories, including apparently unused ones. Both category and reservation deletes are denied in rules independently of the disabled UI control. Disable a category instead. This follows the request's explicit exception because no authoritative product repository/reference invariant exists yet.

The next product milestone should use `products/{id}.categoryId` referencing the stable category ID (currently only an optional TypeScript field). Product creation/reassignment and dependency tracking must be enforced together. A client product query alone would race with concurrent assignments. Keep deletion unavailable until a trusted mechanism proves and preserves zero references during deletion. No existing product records were inspected or migrated.

## Owner release

1. Review the diff and rules against existing live integrations/records. No migration is automatic. Legacy categories must meet the schema and have matching reservations before editing. Enabled documents are public as a whole; do not put private data in them.
2. Publish reviewed rules for **parinaya-silks / (default)** before releasing writes or accepting disabled-category privacy. The previously supplied test-mode grant permitted broad access until 14 November 2026; its current deployed state was not reverified. Do not keep that wildcard alongside these rules or overwrite other live integration rules without reconciliation.

   ```
   npx.cmd firebase deploy --only firestore:rules --project parinaya-silks
   ```

   This owner command was not executed. No index publication is normally needed.
3. Keep the existing Firebase browser variables in ignored `.env.local` and Vercel Production, with project `parinaya-silks` and database `(default)`. Never put service-account credentials in frontend variables, source, Git or logs.
4. Commit/push the reviewed changes through the existing Vercel workflow. Existing hosting rewrites support the new collection/admin routes. Complete hosted acceptance; emulator success does not verify deployment.

## Tests

```
npm.cmd run lint
npm.cmd test
npm.cmd run build
npm.cmd run test:admin
```

`test:admin` starts isolated Auth/Firestore emulators and local browser servers, runs existing authorization plus category tests, then shuts them down. Java, Python Playwright and Chromium are required as in the existing suite. Synthetic records are emulator-only; mutation tests require loopback hosts.

Coverage includes persisted creates/edits/status/order, concurrent duplicate slugs, repeat submissions/idempotency, invalid/bypassed writes, role combinations/revocation, paired reservations, immutable fields, server timestamps, stale edits, public direct/query privacy, blocked deletion, permission errors/retry, not-found/empty states and six viewport widths. Screenshots go to ignored `.cache/categories`.

With ordinary development on 3000 and production preview on 4173:

```
python scripts/storefront-browser.test.py
python scripts/browser-audit.py
python scripts/firebase-browser.test.py
```

General storefront/audit scripts intercept live Firestore requests; real SDK category mutations are verified separately in emulators.

Validation completed on 17 October 2026: typecheck, all 33 unit tests, production build, full admin/category emulator suite, storefront regression checks against local dev and production preview, intercepted Firebase browser checks, and all 192 route/viewport audit checks passed. Mobile and desktop editor screenshots were visually inspected. The build still reports a JavaScript chunk above 500 kB; this is a warning, not a failed check. Hosted behavior and currently deployed rules remain unverified.

## Test one real category locally and on Vercel

After rule publication, the owner can perform these live acceptance steps:

1. Run `npm.cmd run dev` in ordinary mode, open `http://localhost:3000/admin`, sign in and Refresh permissions if necessary. Ordinary mode uses the real configured project; saving is a real mutation. Use the intended real category, not throwaway records.
2. Products & Inventory → Manage Categories → Add category. Enter name/description, review the permanent slug, choose enabled status/order, then Save. Refresh and confirm persistence.
3. In a signed-out browser, open `/collections` and `/collections/{slug}`. Enabled categories appear in saved order with an empty product list. Disable it and confirm it disappears and its URL shows Category not found.
4. Rename/edit/order and refresh; confirm the original URL remains fixed. A duplicate slug must fail. Delete remains unavailable. Leave the category in the owner's desired enabled state.
5. After the owner releases the build, repeat using the SAME category at `https://parinayasilks.vercel.app/admin/categories` and the public collection URLs. Review/test deployed rules for signed-out/customer write denial; a hidden button does not establish database security.

Remaining dependencies: owner rules publication, application release and live acceptance. Safe deletion awaits the product-reference milestone. Product CRUD, uploads, customer login, checkout and payments remain outside scope.